



TECHNOLOGIE JEE

Java Enterprise Edition

Cours Complet + Travaux Pratiques

10 Séances de 3 heures • Licence 3 Informatique

Établissement Supdeco Dakar – School of Engineering & Technology Programme Licence 3 Informatique – Semestre 6	Volume Horaire 10h CM 20h TP Projet Intégré Prérequis Java SE, POO, SQL, Bases de données
---	--

PLAN DU COURS

Séance	Titre	Thème	TP
01	Rappels Java SE & POO Avancée	Java SE	Héritage & polymorphisme
02	Architecture JEE & Serveur Tomcat	Architecture	Déploiement Tomcat
03	Les Servlets HTTP	Servlets	Formulaire CRUD
04	Modèle MVC avec Servlets	MVC	App MVC complète
05	Pages JSP & JSTL	JSP/JSTL	Interface JSP
06	Connexion JDBC	JDBC	Accès base de données
07	ORM & JPA/Hibernate	ORM	Mapping entités
08	Introduction Spring Boot	Spring Boot	Premier projet SB
09	Spring Boot REST API	REST API	API CRUD complète

10	Projet Intégré SpringBoot	Projet	Application finale
----	---------------------------	--------	--------------------

SÉANCE
01/10
3 h

Technologie JEE – L3 Informatique

Rappels Java SE & Programmation Orientée
Objet Avancée**Objectifs de la séance**

- › Maîtriser la structure d'un projet Java (packages, classes, interfaces)
- › Appliquer les principes avancés de la POO : héritage, polymorphisme, abstraction
- › Utiliser les collections Java (List, Map, Set)
- › Préparer les bases pour le développement JEE

PARTIE THÉORIQUE**1.1 Structure d'un projet Java**

Un projet Java est organisé en packages, qui correspondent à des répertoires dans le système de fichiers. Cette organisation permet de regrouper les classes par fonctionnalité et d'éviter les conflits de noms.

```
// Structure d'un projet Java
mon_projet/
├── src/
│   ├── sn/
│   │   ├── supdeco/
│   │   │   ├── modele/
│   │   │   │   ├── Etudiant.java
│   │   │   │   ├── Cours.java
│   │   │   │   ├── service/
│   │   │   │   │   ├── EtudiantService.java
│   │   │   │   │   └── Main.java
│   │   └── lib/
```

1.2 Encapsulation & Getters/Setters

L'encapsulation consiste à masquer les attributs d'une classe et à n'y accéder que via des méthodes publiques.

```
package sn.supdeco.modele;

public class Etudiant {
    // Attributs privés (encapsulés)
    private int id;
    private String nom;
    private String prenom;
    private double moyenne;

    // Constructeur
    public Etudiant(int id, String nom, String prenom) {
        this.id = id;
        this.nom = nom;
    }
}
```

```
        this.prenom = prenom;
        this.moyenne = 0.0;
    }

    // Getter et Setter
    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public double getMoyenne() { return moyenne; }
    public void setMoyenne(double moyenne) {
        if (moyenne >= 0 && moyenne <= 20)
            this.moyenne = moyenne;
    }

    @Override
    public String toString() {
        return prenom + " " + nom + " (" + moyenne + "/20)";
    }
}
```

1.3 Héritage et Polymorphisme

L'héritage permet de créer une nouvelle classe à partir d'une classe existante. Le polymorphisme permet d'utiliser une référence de type parent pour pointer vers un objet enfant.

```
// Classe abstraite parente
public abstract class Personne {
    protected String nom;
    protected String email;

    public Personne(String nom, String email) {
        this.nom = nom;
        this.email = email;
    }

    // Méthode abstraite - doit être redéfinie dans les sous-classes
    public abstract String getRole();

    public void afficherInfos() {
        System.out.println("[ " + getRole() + " ] " + nom + " <" + email + ">");
    }
}

// Sous-classe Etudiant
public class Etudiant extends Personne {
    private String matricule;

    public Etudiant(String nom, String email, String matricule) {
        super(nom, email); // appel constructeur parent
        this.matricule = matricule;
    }

    @Override
    public String getRole() { return "Étudiant"; }
}

// Sous-classe Enseignant
public class Enseignant extends Personne {
    private String specialite;

    public Enseignant(String nom, String email, String specialite) {
```

```
        super(nom, email);
        this.specialite = specialite;
    }

    @Override
    public String getRole() { return "Enseignant - " + specialite; }
}
```

1.4 Interfaces

Une interface définit un contrat que les classes doivent respecter. Elle ne contient que des méthodes abstraites (Java < 8) ou des méthodes par défaut (Java ≥ 8).

```
public interface Persistable {
    void sauvegarder();
    void supprimer();
    Object charger(int id);
}

// Une classe peut implémenter plusieurs interfaces
public class EtudiantDAO implements Persistable {
    @Override
    public void sauvegarder() {
        System.out.println("Étudiant sauvegardé en BDD");
    }
    // ... autres méthodes
}
```

1.5 Collections Java

```
import java.util.*;

public class ExempleCollections {
    public static void main(String[] args) {
        // ArrayList - liste ordonnée avec doublons
        List<Etudiant> etudiants = new ArrayList<>();
        etudiants.add(new Etudiant(1, "Diallo", "Moussa"));
        etudiants.add(new Etudiant(2, "Ndiaye", "Fatou"));

        // Parcours avec for-each
        for (Etudiant e : etudiants) {
            System.out.println(e);
        }

        // HashMap - paires clé/valeur
        Map<Integer, String> notes = new HashMap<>();
        notes.put(1, "A");
        notes.put(2, "B+");
        System.out.println(notes.get(1)); // Affiche: A
    }
}
```

Durée : 45 minutes | Rendu : fichiers .java compressés en .zip

Contexte

Vous allez développer un mini-système de gestion des étudiants de Supdeco en Java SE, en appliquant les concepts de POO vus en cours.

Travail demandé

1. Créez un package `sn.supdeco.gestion` avec les classes suivantes :

- › `Personne` (abstraite) : attributs `nom`, `prenom`, `email` ; méthode abstraite `getDescription()`
- › `Etudiant` (hérite de `Personne`) : attributs `matricule`, `filiere`, `annee` ; implémente `getDescription()`
- › `Enseignant` (hérite de `Personne`) : attributs `grade`, `specialite` ; implémente `getDescription()`

2. Créez une interface `IGestionnable` avec les méthodes : `ajouter()`, `supprimer(int id)`, `lister()`

3. Créez une classe `EtudiantService` qui implémente `IGestionnable` avec une `ArrayList<Etudiant>`

4. Dans la classe `Main`, créez au moins 5 étudiants, ajoutez-les, affichez la liste, puis supprimez un étudiant par son matricule

5. **BONUS** : Triez la liste des étudiants par nom (utilisez `Collections.sort()` et `Comparable` ou `Comparator`)

Critères d'évaluation :

- › Bonne utilisation de l'héritage et du polymorphisme (5 pts)
- › Implémentation correcte de l'interface (3 pts)
- › Code propre, commenté, packages respectés (2 pts)

SÉANCE
02/10
2h

Technologie JEE – L3 Informatique Architecture JEE & Serveur Tomcat

Objectifs de la séance

- › Comprendre l'architecture JEE et ses composants
- › Installer et configurer Apache Tomcat
- › Créer et déployer une application web Java
- › Comprendre la structure d'un projet web Java (dossier WEB-INF, web.xml)

PARTIE THÉORIQUE

2.1 Architecture JEE

JEE (Jakarta Enterprise Edition, anciennement Java EE) est une plateforme de développement d'applications d'entreprise basée sur Java. Elle définit un ensemble de spécifications pour développer des applications web robustes, sécurisées et distribuées.

Les couches de l'architecture JEE :

- › Couche Présentation : JSP, HTML, CSS, JavaScript
- › Couche Contrôle : Servlets (traitent les requêtes HTTP)
- › Couche Métier : EJB, Spring Beans (logique applicative)
- › Couche Données : JPA, JDBC (accès aux données)

2.2 Apache Tomcat – Le conteneur de Servlets

Apache Tomcat est un serveur d'applications open source qui implémente les spécifications Servlet et JSP. Il joue le rôle de conteneur web pour les applications JEE légères.

```
# Télécharger Tomcat 10.x depuis https://tomcat.apache.org
# Structure du répertoire Tomcat
apache-tomcat-10.1.x/
├── bin/           → scripts de démarrage (startup.sh / startup.bat)
├── conf/          → fichiers de configuration (server.xml, web.xml)
├── webapps/       → dossier de déploiement des applications
├── logs/          → journaux du serveur
└── lib/           → bibliothèques partagées
```

2.3 Structure d'une application web Java

```
mon_app_web/
├── src/
│   └── main/
│       └── java/
```



2.4 Le fichier web.xml

Le fichier web.xml (Deployment Descriptor) configure les servlets, les filtres, et les paramètres de l'application.

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  version="5.0">

  <!-- Déclaration d'une Servlet -->
  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>sn.supdeco>HelloServlet</servlet-class>
  </servlet>

  <!-- Mapping URL → Servlet -->
  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>

  <!-- Page d'accueil -->
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>

</web-app>

```

2.5 Ma première application web

```

package sn.supdeco;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

// Annotation moderne - remplace web.xml pour la servlet
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

```



```
resp.setContentType("text/html;charset=UTF-8");
PrintWriter out = resp.getWriter();

out.println("<!DOCTYPE html>");
out.println("<html><head><title>Supdeco JEE</title></head><body>");
out.println("<h1>Bienvenue à Supdeco Dakar!</h1>");
out.println("<p>Votre première application JEE fonctionne.</p>");
out.println("</body></html>");
}
```

✦ TP 02

Déploiement de la première application web sur Tomcat

Durée : 50 minutes | Rendu : capture écran + fichiers du projet

Travail demandé

6. Installer Apache Tomcat 10.x sur votre machine (ou utiliser IntelliJ IDEA avec Tomcat intégré)
7. Créer un projet Maven Dynamic Web avec la structure correcte
8. Créer une Servlet 'AccueilServlet' mappée sur /accueil qui affiche une page HTML avec :

- › Le logo Supdeco (image)
- › Le nom et prénom de l'étudiant
- › La date et l'heure actuelles (utilisez java.time.LocalDateTime)
- › Un lien de retour vers la page d'accueil

9. Configurer le web.xml correctement
10. Déployer sur Tomcat et tester dans le navigateur
11. **BONUS :** Créer une deuxième servlet 'InfoServlet' qui affiche les informations du serveur (ServerInfo, Java version)

SÉANCE
03/10
2h

Technologie JEE – L3 Informatique
Les Servlets HTTP

🎯 Objectifs de la séance

- › Comprendre le cycle de vie d'une Servlet
 - › Maîtriser les méthodes doGet, doPost, doPut, doDelete
 - › Lire les paramètres de requête et les données de formulaire
 - › Gérer les sessions et les cookies
-
- ›

PARTIE THÉORIQUE

3.1 Cycle de vie d'une Servlet

1. init() → appelée une fois au chargement (initialisation)
2. service() → appelée à chaque requête (redirige vers doGet/doPost...)
3. destroy() → appelée à l'arrêt du serveur (nettoyage)

```
@WebServlet("/produits")
public class ProduitServlet extends HttpServlet {

    private List<String> produits;

    @Override
    public void init() throws ServletException {
        // Initialisation au démarrage
        produits = new ArrayList<>();
        produits.add("Ordinateur HP");
        produits.add("Tablette Samsung");
        System.out.println("ProduitServlet initialisée");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        // Lecture d'un paramètre de l'URL (?action=liste)
        String action = req.getParameter("action");
        resp.setContentType("text/html;charset=UTF-8");
        PrintWriter out = resp.getWriter();

        if ("liste".equals(action)) {
            out.println("<ul>");
            for (String p : produits) {
                out.println("<li>" + p + "</li>");
            }
            out.println("</ul>");
        }
    }
}
```

```
    }  
}  
  
@Override  
protected void doPost(HttpServletRequest req, HttpServletResponse resp)  
    throws ServletException, IOException {  
    // Lecture des données d'un formulaire HTML  
    req.setCharacterEncoding("UTF-8");  
    String nomProduit = req.getParameter("nom");  
  
    if (nomProduit != null && !nomProduit.isEmpty()) {  
        produits.add(nomProduit);  
        resp.sendRedirect(req.getContextPath() + "/produits?action=liste");  
    }  
}  
  
@Override  
public void destroy() {  
    System.out.println("ProduitServlet détruite - " + produits.size() + "  
produits");  
}  
}
```

3.2 Gestion des Sessions

Une session HTTP permet de maintenir l'état entre plusieurs requêtes d'un même utilisateur.

```
// Créer ou récupérer la session  
HttpSession session = req.getSession();  
  
// Stocker des données en session  
session.setAttribute("utilisateur", "Moussa Diallo");  
session.setAttribute("role", "admin");  
session.setMaxInactiveInterval(30 * 60); // 30 minutes  
  
// Lire depuis la session  
String user = (String) session.getAttribute("utilisateur");  
  
// Invalider la session (déconnexion)  
session.invalidate();
```

3.3 Redirection et Forwarding

```
// Redirection côté client (change l'URL dans le navigateur)  
resp.sendRedirect("http://www.supdeco.sn");  
resp.sendRedirect(req.getContextPath() + "/accueil");  
  
// Forward côté serveur (l'URL ne change pas, transmet req/resp)  
RequestDispatcher rd = req.getRequestDispatcher("/WEB-INF/vues/liste.jsp");  
rd.forward(req, resp);  
  
// Passer des données vers la JSP via les attributs de requête  
req.setAttribute("listeProduits", produits);  
rd.forward(req, resp);
```

✦ TP 03

Application de Gestion de Contacts avec Formulaire

Durée : 50 minutes | Rendu : projet complet zippé

Contexte

Créer une application web permettant de gérer une liste de contacts (CRUD simplifié en mémoire).

Travail demandé

12. Créez une classe Contact (nom, prenom, telephone, email) avec getters/setters
13. Créez une ContactServlet qui gère les actions suivantes via le paramètre 'action' :

- › action=liste → affiche tous les contacts en HTML
- › action=ajouter → affiche le formulaire d'ajout
- › doPost() → traite le formulaire et redirige vers la liste
- › action=supprimer&id=X → supprime le contact numéro X

14. Stockez les contacts dans la session HTTP (persistance pendant la session)
15. Gérez l'encodage UTF-8 pour les prénoms avec accents (Ibrahima, Aïssatou...)
16. Ajoutez une validation simple : tous les champs sont obligatoires
17. BONUS : Ajoutez la fonctionnalité de recherche par nom

SÉANCE
04/10
2h

Technologie JEE – L3 Informatique
Modèle MVC avec Servlets

🎯 Objectifs de la séance

- › Comprendre et appliquer le patron de conception MVC (Model-View-Controller)
- › Séparer les responsabilités : données, présentation, contrôle
- › Organiser un projet JEE selon l'architecture MVC
- › Utiliser le RequestDispatcher pour transférer vers les vues

PARTIE THÉORIQUE

4.1 Le patron MVC

Modèle (Model) → Représente les données et la logique métier (classes Java, DAO)

Vue (View) → Affichage des données à l'utilisateur (JSP, HTML)

Contrôleur (Controller) → Reçoit les requêtes, appelle le modèle, choisit la vue (Servlet)

4.2 Structure du projet MVC

```
src/
├── java/sn/supdeco/
│   ├── modele/
│   │   ├── Etudiant.java      ← Modèle (entité)
│   │   └── EtudiantDAO.java   ← Accès aux données
│   ├── service/
│   │   └── EtudiantService.java ← Logique métier
│   └── controleur/
│       └── EtudiantControleur.java ← Servlet (contrôleur)
├── webapp/
│   ├── WEB-INF/
│   │   └── vues/
│   │       ├── liste_etudiants.jsp ← Vue liste
│   │       ├── form_etudiant.jsp   ← Vue formulaire
│   │       └── detail_etudiant.jsp  ← Vue détail
│   └── index.html
```

4.3 Le Contrôleur (Servlet)

```
@WebServlet("/etudiants")
public class EtudiantControleur extends HttpServlet {

    private EtudiantService service;
```

```

@Override
public void init() {
    service = new EtudiantService();
}

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {

    String action = req.getParameter("action");
    if (action == null) action = "liste";

    switch (action) {
        case "liste":
            listerEtudiants(req, resp);
            break;
        case "nouveau":
            afficherFormulaire(req, resp);
            break;
        case "supprimer":
            supprimerEtudiant(req, resp);
            break;
        default:
            resp.sendError(HttpServletResponse.SC_NOT_FOUND);
    }
}

private void listerEtudiants(HttpServletRequest req, HttpServletResponse
resp)
    throws ServletException, IOException {
    List<Etudiant> etudiants = service.listerTous();
    req.setAttribute("etudiants", etudiants); // Envoie au Modèle vers la
Vue
    req.getRequestDispatcher("/WEB-INF/vues/liste_etudiants.jsp")
        .forward(req, resp);
}
}

```

4.4 Le Modèle (Service + DAO)

```

// Service - Logique métier
public class EtudiantService {
    private EtudiantDAO dao = new EtudiantDAO();

    public List<Etudiant> listerTous() {
        return dao.findAll();
    }

    public boolean ajouter(Etudiant e) {
        // Validation métier
        if (e.getNom() == null || e.getNom().isEmpty()) return false;
        dao.save(e);
        return true;
    }
}

// DAO - Accès aux données (en mémoire pour l'instant)
public class EtudiantDAO {
    private static List<Etudiant> bdd = new ArrayList<>();
    private static int compteur = 1;

    public List<Etudiant> findAll() {

```

```
        return new ArrayList<>(bdd);  
    }  
  
    public void save(Etudiant e) {  
        e.setId(compteur++);  
        bdd.add(e);  
    }  
}
```

◆ TP 04

Application MVC – Gestion des Cours de Supdeco

Durée : 50 minutes | Rendu : projet structuré MVC zippé

Travail demandé

18. Créez une application MVC complète pour gérer les cours de Supdeco avec les classes : Cours (id, code, intitule, credits, enseignant), CoursDAO (en mémoire), CoursService, CoursControleur
19. Le contrôleur doit gérer les actions : liste, nouveau, sauvegarder (doPost), modifier, supprimer
20. Créez les vues JSP : liste_cours.jsp, formulaire_cours.jsp
21. La vue liste doit afficher un tableau HTML avec les cours, et des boutons Modifier/Supprimer
22. BONUS : Ajoutez la pagination (5 cours par page) dans la vue liste

SÉANCE
05/10
2h

Technologie JEE – L3 Informatique Pages JSP & JSTL

Objectifs de la séance

- › Créer et utiliser des pages JSP (JavaServer Pages)
- › Utiliser les expressions EL (Expression Language)
- › Maîtriser les balises JSTL (forEach, if, choose, fmt)
- › Créer des layouts réutilisables avec include et taglib

PARTIE THÉORIQUE

5.1 Syntaxe JSP de base

```
<%-- Commentaire JSP (non visible dans le HTML généré) --%>
<%@ page contentType="text/html;charset=UTF-8" language="java" %>

<!-- Scriptlet : code Java dans JSP (déconseillé aujourd'hui) -->
<% String message = "Bonjour"; %>

<!-- Expression : affiche la valeur -->
<p><%= message %></p>

<!-- Expression Language (EL) - moderne et recommandé -->
<p>${message}</p>
<p>Etudiant : ${etudiant.nom} ${etudiant.prenom}</p>
<p>Nombre : ${liste.size()}</p>
```

5.2 JSTL – Les balises essentielles

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<!-- Boucle forEach -->
<table class="table">
  <c:forEach var="etudiant" items="${etudiants}" varStatus="status">
    <tr class="${status.index % 2 == 0 ? 'pair' : 'impair'}">
      <td>${status.index + 1}</td>
      <td>${etudiant.nom}</td>
      <td>${etudiant.prenom}</td>
      <td>
        <!-- Formatage de date -->
        <fmt:formatDate value="${etudiant.dateNaissance}" pattern="dd/MM/yyyy"/>
      </td>
    </tr>
  </c:forEach>
</table>

<!-- Condition if -->
<c:if test="${etudiants.size() == 0}">
  <p class="alert">Aucun étudiant trouvé.</p>
```



```
</c:if>

<!-- Condition choose/when/otherwise -->
<c:choose>
  <c:when test="\${etudiant.moyenne} >= 12">
    <span class="badge vert">Admis</span>
  </c:when>
  <c:when test="\${etudiant.moyenne} >= 10">
    <span class="badge orange">Passable</span>
  </c:when>
  <c:otherwise>
    <span class="badge rouge">Ajourné</span>
  </c:otherwise>
</c:choose>

<!-- URL avec paramètres -->
<a href="\<c:url value='/etudiants'><c:param name='action'
value='modifier'>/><c:param name='id' value='\${etudiant.id}'>/></c:url">
  Modifier
</a>
```

5.3 Include et Layout

```
<%-- header.jsp - réutilisable --%>
<header>
  
  <nav>
    <a
  href="\${pageContext.request.contextPath}/etudiants?action=liste">Étudiants</a>
    <a href="\${pageContext.request.contextPath}/cours?action=liste">Cours</a>
  </nav>
</header>

<%-- Dans votre page JSP principale --%>
<%@ include file="/WEB-INF/vues/commun/header.jsp" %>
<!-- contenu de la page -->
<%@ include file="/WEB-INF/vues/commun/footer.jsp" %>
```

✦ TP 05

Interface JSP complète avec Bootstrap 5

Durée : 50 minutes | Rendu : pages JSP du projet

Travail demandé

23. Reprenez le projet MVC de la séance 4 et remplacez la génération HTML dans les Servlets par des pages JSP
24. Créez un layout commun (header.jsp + footer.jsp) avec Bootstrap 5 (CDN)
25. La page liste_cours.jsp doit afficher :

- › Un tableau Bootstrap avec les cours (rayures alternées)
- › Des badges colorés pour les crédits (vert ≥ 3 , orange = 2, rouge = 1)
- › Des boutons d'action avec icônes Bootstrap Icons

-
26. Créez une page form_cours.jsp avec validation côté client (required, minlength)
 27. Ajoutez des messages flash (succès/erreur) après les opérations
 28. BONUS : Ajoutez une barre de recherche avec filtre côté client (JavaScript)

SÉANCE
06/10
2h

Technologie JEE – L3 Informatique

Connexion JDBC – Accès aux Bases de Données

Objectifs de la séance

- › Comprendre l'architecture JDBC et ses composants
- › Établir une connexion à une base de données MySQL/PostgreSQL
- › Exécuter des requêtes SQL (CRUD) avec JDBC
- › Utiliser les PreparedStatement pour éviter les injections SQL
- › Implémenter le pattern DAO avec JDBC

PARTIE THÉORIQUE

6.1 Architecture JDBC

JDBC (Java Database Connectivity) est l'API Java standard pour interagir avec des bases de données relationnelles. Elle fonctionne avec n'importe quel SGBD via un pilote (driver) spécifique.

Les étapes d'une connexion JDBC :

- › 1. Charger le driver (automatique avec JDBC 4.0+)
- › 2. Ouvrir la connexion (DriverManager.getConnection)
- › 3. Créer un Statement ou PreparedStatement
- › 4. Exécuter la requête SQL
- › 5. Traiter le ResultSet
- › 6. Fermer les ressources (try-with-resources)

6.2 Configuration de la base de données

```
-- Script SQL pour MySQL
CREATE DATABASE supdeco_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE supdeco_db;

CREATE TABLE etudiants (
    id          INT AUTO INCREMENT PRIMARY KEY,
    matricule   VARCHAR(20) UNIQUE NOT NULL,
    nom         VARCHAR(100) NOT NULL,
    prenom      VARCHAR(100) NOT NULL,
    email       VARCHAR(150) UNIQUE,
    filiere     VARCHAR(50),
    moyenne    DECIMAL(4,2) DEFAULT 0.00,
    date_inscription TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO etudiants (matricule, nom, prenom, email, filiere)
VALUES ('L3INFO001', 'DIALLO', 'Moussa', 'moussa@supdeco.sn', 'L3
Informatique');
```

```
('L3INFO002', 'NDIAYE', 'Fatou', 'fatou@supdeco.sn', 'L3 Informatique');
```

6.3 Classe de Connexion (Singleton)

```
public class DatabaseConfig {
    private static final String URL =

    "jdbc:mysql://localhost:3306/supdeco_db?useSSL=false&serverTimezone=Africa/Dakar
    &characterEncoding=utf8";
    private static final String USER = "root";
    private static final String PASSWORD = "votre_mdp";

    // Pattern Singleton pour la connexion
    private static Connection connexion = null;

    public static Connection getConnexion() throws SQLException {
        if (connexion == null || connexion.isClosed()) {
            connexion = DriverManager.getConnection(URL, USER, PASSWORD);
        }
        return connexion;
    }
}
```

6.4 DAO avec PreparedStatement

```
public class EtudiantDAO {

    // Lire tous les étudiants
    public List<Etudiant> findAll() throws SQLException {
        List<Etudiant> liste = new ArrayList<>();
        String sql = "SELECT * FROM etudiants ORDER BY nom";

        try (Connection cn = DatabaseConfig.getConnexion();
            Statement st = cn.createStatement();
            ResultSet rs = st.executeQuery(sql)) {

            while (rs.next()) {
                Etudiant e = new Etudiant();
                e.setId(rs.getInt("id"));
                e.setMatricule(rs.getString("matricule"));
                e.setNom(rs.getString("nom"));
                e.setPrenom(rs.getString("prenom"));
                e.setMoyenne(rs.getDouble("moyenne"));
                liste.add(e);
            }
        }
        return liste;
    }

    // Insérer un étudiant (PreparedStatement - sécurisé !)
    public boolean save(Etudiant e) throws SQLException {
        String sql = "INSERT INTO etudiants (matricule, nom, prenom, email,
        filiere) VALUES (?, ?, ?, ?, ?)";

        try (Connection cn = DatabaseConfig.getConnexion();
            PreparedStatement ps = cn.prepareStatement(sql)) {

            ps.setString(1, e.getMatricule());
            ps.setString(2, e.getNom());
            ps.setString(3, e.getPrenom());
        }
    }
}
```

```

        ps.setString(4, e.getEmail());
        ps.setString(5, e.getFiliere());
        return ps.executeUpdate() > 0;
    }
}

// Mettre à jour
public boolean update(Etudiant e) throws SQLException {
    String sql = "UPDATE etudiants SET nom=?, prenom=?, email=?, moyenne=?
WHERE id=?";
    try (Connection cn = DatabaseConfig.getConnexion();
        PreparedStatement ps = cn.prepareStatement(sql)) {
        ps.setString(1, e.getNom());
        ps.setString(2, e.getPrenom());
        ps.setString(3, e.getEmail());
        ps.setDouble(4, e.getMoyenne());
        ps.setInt(5, e.getId());
        return ps.executeUpdate() > 0;
    }
}

// Supprimer
public boolean delete(int id) throws SQLException {
    String sql = "DELETE FROM etudiants WHERE id = ?";
    try (Connection cn = DatabaseConfig.getConnexion();
        PreparedStatement ps = cn.prepareStatement(sql)) {
        ps.setInt(1, id);
        return ps.executeUpdate() > 0;
    }
}
}

```

✦ TP 06

Application CRUD Étudiants connectée à MySQL

Durée : 50 minutes | Rendu : projet + script SQL

Travail demandé

29. Créez la base de données supdeco_db avec les tables etudiants et filieres
30. Ajoutez le driver MySQL (mysql-connector-j) au projet via Maven (pom.xml)
31. Implémentez le EtudiantDAO complet avec les 4 opérations CRUD
32. Connectez le DAO au Service et au Contrôleur du projet MVC
33. Testez toutes les opérations (ajouter, lister, modifier, supprimer) en base de données réelle
34. BONUS : Gérez les exceptions SQLException avec des messages d'erreur clairs affichés à l'utilisateur

SÉANCE
07/10
2h

Technologie JEE – L3 Informatique
ORM & JPA/Hibernate

🎯 Objectifs de la séance

- › Comprendre le concept d'ORM (Object-Relational Mapping)
- › Configurer JPA avec Hibernate comme implémentation
- › Annoter les entités Java pour le mapping objet-relationnel
- › Utiliser l'EntityManager pour les opérations CRUD
- › Écrire des requêtes JPQL

PARTIE THÉORIQUE

7.1 Qu'est-ce que l'ORM ?

L'ORM (Object-Relational Mapping) est une technique qui permet de mapper automatiquement les objets Java sur des tables de base de données, sans écrire de SQL manuellement.

Avantages :

- › Productivité : plus besoin d'écrire du SQL répétitif
- › Portabilité : changez de SGBD sans modifier le code Java
- › Sécurité : protection automatique contre les injections SQL
- › Maintenance : le code métier est séparé des requêtes

7.2 Configuration pom.xml

```
<dependencies>
  <!-- JPA API -->
  <dependency>
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
    <version>3.1.0</version>
  </dependency>
  <!-- Hibernate (implémentation JPA) -->
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.4.0.Final</version>
  </dependency>
  <!-- Connecteur MySQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
  </dependency>
</dependencies>
```

7.3 Annotation d'une Entité JPA

```
package sn.supdeco.entite;

import jakarta.persistence.*;

@Entity // Cette classe est une entité JPA
@Table(name = "etudiants") // Nom de la table en BDD
public class Etudiant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "matricule", unique = true, nullable = false, length = 20)
    private String matricule;

    @Column(nullable = false, length = 100)
    private String nom;

    @Column(nullable = false, length = 100)
    private String prenom;

    @Column(unique = true)
    private String email;

    @ManyToOne // Relation Many-to-One
    @JoinColumn(name = "filiere_id")
    private Filiere filiere;

    // Constructeurs, getters, setters...
}

@Entity
@Table(name = "filiere")
public class Filiere {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String code;
    private String libelle;

    @OneToMany(mappedBy = "filiere") // Relation inverse
    private List<Etudiant> etudiants = new ArrayList<>();
}
```

7.4 EntityManager – CRUD JPA

```
public class EtudiantRepository {

    private EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("supdecoPU");

    public List<Etudiant> findAll() {
        EntityManager em = emf.createEntityManager();
        try {
            // JPQL (Java Persistence Query Language)
            return em.createQuery("SELECT e FROM Etudiant e ORDER BY e.nom",
                Etudiant.class);
        }
    }
}
```

```
        .getResultList();
    } finally { em.close(); }
}

public Etudiant findById(int id) {
    EntityManager em = emf.createEntityManager();
    try { return em.find(Etudiant.class, id); }
    finally { em.close(); }
}

public void save(Etudiant e) {
    EntityManager em = emf.createEntityManager();
    em.getTransaction().begin();
    try {
        em.persist(e);          // INSERT
        em.getTransaction().commit();
    } catch (Exception ex) {
        em.getTransaction().rollback();
        throw ex;
    } finally { em.close(); }
}

public void update(Etudiant e) {
    EntityManager em = emf.createEntityManager();
    em.getTransaction().begin();
    try {
        em.merge(e);           // UPDATE
        em.getTransaction().commit();
    } finally { em.close(); }
}
}
```

◆ TP 07

Mapping JPA – Application Supdeco avec Relations

Durée : 50 minutes | Rendu : entités annotées + persistence.xml

Travail demandé

35. Créez les entités JPA annotées : Etudiant, Filiere, Cours, Inscription
36. Définissez les relations JPA : Etudiant @ManyToOne Filiere, Inscription @ManyToOne Etudiant et @ManyToOne Cours
37. Configurez persistence.xml avec hibernate.hbm2ddl.auto=update (génération auto de tables)
38. Créez un EtudiantRepository avec findAll(), findById(), findByFiliere(String code)
39. Testez avec une classe Main : créez 3 filieres, 10 étudiants, 5 cours, inscrivez des étudiants
40. BONUS : Écrivez une requête JPQL qui retourne la moyenne des étudiants par filière

SÉANCE
08/10
2h

Technologie JEE – L3 Informatique
Introduction à Spring Boot

🎯 Objectifs de la séance

- › Comprendre l'écosystème Spring et l'intérêt de Spring Boot
- › Créer un projet Spring Boot avec Spring Initializr
- › Comprendre les annotations essentielles (@Component, @Service, @Repository, @Controller)
- › Configurer application.properties
- › Démarrer une application Spring Boot

PARTIE THÉORIQUE

8.1 Spring vs Spring Boot

Spring Framework : framework complet mais complexe à configurer (XML ou Java Config)

Spring Boot : surcouche qui simplifie Spring avec la philosophie 'Convention over Configuration' — zéro XML, configuration automatique, serveur embarqué

8.2 Créer un projet Spring Boot

Accédez à <https://start.spring.io> et configurez :

```
Project : Maven
Language : Java
Spring Boot : 3.x.x
Group : sn.supdeco
Artifact : gestion-app
Packaging : Jar
Java : 17 ou 21
```

Dépendances à ajouter :

- ✓ Spring Web
- ✓ Spring Data JPA
- ✓ MySQL Driver (ou H2 pour les tests)
- ✓ Thymeleaf (moteur de template)
- ✓ Spring Boot DevTools
- ✓ Lombok (optionnel, réduit le code)

8.3 Structure d'un projet Spring Boot

```
gestion-app/
├── src/main/java/sn/supdeco/gestionapp/
```

```

├── GestionAppApplication.java ← Point d'entrée @SpringBootApplication
├── entite/
│   └── Etudiant.java ← @Entity
├── repository/
│   └── EtudiantRepository.java ← @Repository, extends JpaRepository
├── service/
│   └── EtudiantService.java ← @Service
├── controleur/
│   └── EtudiantControleur.java ← @Controller ou @RestController
├── src/main/resources/
│   ├── application.properties ← Configuration
│   ├── templates/ ← Vues Thymeleaf
│   │   └── etudiants/
│   │       └── liste.html
└── pom.xml

```

8.4 application.properties

```

# Configuration de la base de données
spring.datasource.url=jdbc:mysql://localhost:3306/supdeco_db
spring.datasource.username=root
spring.datasource.password=votre_mdp
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

# Port du serveur
server.port=8080
server.servlet.context-path=/supdeco

# Encodage
spring.web.resources.charset=UTF-8

```

8.5 Les Annotations Essentielles

Annotation	Rôle
@SpringBootApplication	Point d'entrée – active l'auto-configuration et le scan des composants
@Entity	Marque une classe Java comme entité JPA (table en BDD)
@Repository	Marque une interface comme couche d'accès aux données (DAO)
@Service	Marque une classe comme service de logique métier
@Controller	Contrôleur web MVC (retourne des vues HTML)
@RestController	Contrôleur REST (retourne du JSON/XML)
@Autowired	Injection automatique de dépendances (IoC)
@GetMapping	Mappe une méthode sur une requête HTTP GET
@PostMapping	Mappe une méthode sur une requête HTTP POST

```
@SpringBootApplication
public class GestionAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(GestionAppApplication.class, args);
        // Le serveur Tomcat embarqué démarre automatiquement
        // http://localhost:8080/supdeco
    }
}
```

✦ TP 08

Premier projet Spring Boot – Hello Supdeco

Durée : 50 minutes | Rendu : projet Maven complet

Travail demandé

41. Créez un projet Spring Boot via start.spring.io avec les dépendances : Web, Thymeleaf, DevTools
42. Créez un AccueilController (@Controller) avec une méthode mappée sur GET /accueil
43. La méthode ajoute au model : titre, auteur (votre nom), date, listeCours (ArrayList de String)
44. Créez la vue Thymeleaf accueil.html avec : logo, tableau des cours avec th:each, condition th:if
45. Ajoutez une deuxième route GET /contact qui affiche un formulaire de contact
46. BONUS : Créez un EtudiantController avec une route /etudiants qui affiche une liste d'objets Etudiant

SÉANCE
09/10
2h

Technologie JEE – L3 Informatique

Spring Boot – REST API Complète

🎯 Objectifs de la séance

- › Comprendre l'architecture REST et ses principes
- › Créer une API REST complète avec Spring Boot
- › Utiliser Spring Data JPA (JpaRepository) pour simplifier l'accès aux données
- › Gérer les réponses HTTP (statuts, corps JSON)
- › Tester l'API avec Postman ou cURL

PARTIE THÉORIQUE

9.1 Principes REST

REST (Representational State Transfer) est un style d'architecture pour créer des API web.

- › GET /etudiants → Lister tous les étudiants
- › GET /etudiants/{id} → Obtenir un étudiant par ID
- › POST /etudiants → Créer un nouvel étudiant
- › PUT /etudiants/{id} → Modifier un étudiant
- › DELETE /etudiants/{id} → Supprimer un étudiant

9.2 Spring Data JPA – JpaRepository

```
package sn.supdeco.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

@Repository
public interface EtudiantRepository extends JpaRepository<Etudiant, Integer> {
    // Spring Data génère automatiquement l'implémentation !

    // Méthodes gratuites : findAll(), findById(), save(), delete()...

    // Requêtes par convention de nommage :
    List<Etudiant> findByNomContainingIgnoreCase(String nom);
    List<Etudiant> findByFiliereCode(String code);
    Optional<Etudiant> findByMatricule(String matricule);

    // Requête JPQL personnalisée
```

```
@Query("SELECT e FROM Etudiant e WHERE e.moyenne >= :seuil ORDER BY  
e.moyenne DESC")  
List<Etudiant> findByMoyenneSuperieure(@Param("seuil") double seuil);  
}
```

9.3 Service Layer

```
@Service  
public class EtudiantService {  
  
    @Autowired // Injection automatique du repository  
    private EtudiantRepository repository;  
  
    public List<Etudiant> listerTous() {  
        return repository.findAll();  
    }  
  
    public Etudiant trouverParId(int id) {  
        return repository.findById(id)  
            .orElseThrow(() -> new RuntimeException("Étudiant non trouvé: " +  
id));  
    }  
  
    public Etudiant creer(Etudiant e) {  
        return repository.save(e); // INSERT si id=null, UPDATE sinon  
    }  
  
    public Etudiant modifier(int id, Etudiant e) {  
        Etudiant existant = trouverParId(id);  
        existant.setNom(e.getNom());  
        existant.setPrenom(e.getPrenom());  
        existant.setEmail(e.getEmail());  
        return repository.save(existant);  
    }  
  
    public void supprimer(int id) {  
        repository.deleteById(id);  
    }  
}
```

9.4 RestController CRUD complet

```
@RestController  
@RequestMapping("/api/etudiants")  
public class EtudiantRestControleur {  
  
    @Autowired  
    private EtudiantService service;  
  
    // GET /api/etudiants  
    @GetMapping  
    public ResponseEntity<List<Etudiant>> listerTous() {  
        return ResponseEntity.ok(service.listerTous());  
    }  
  
    // GET /api/etudiants/5  
    @GetMapping("/{id}")  
    public ResponseEntity<Etudiant> trouverParId(@PathVariable int id) {  
        try {
```

```

        return ResponseEntity.ok(service.trouverParId(id));
    } catch (RuntimeException e) {
        return ResponseEntity.notFound().build(); // 404
    }
}

// POST /api/etudiants
@PostMapping
public ResponseEntity<Etudiant> creer(@RequestBody Etudiant etudiant) {
    Etudiant cree = service.creer(etudiant);
    URI location = URI.create("/api/etudiants/" + cree.getId());
    return ResponseEntity.created(location).body(cree); // 201 Created
}

// PUT /api/etudiants/5
@PutMapping("/{id}")
public ResponseEntity<Etudiant> modifier(@PathVariable int id, @RequestBody
Etudiant e) {
    return ResponseEntity.ok(service.modifier(id, e));
}

// DELETE /api/etudiants/5
@DeleteMapping("/{id}")
public ResponseEntity<Void> supprimer(@PathVariable int id) {
    service.supprimer(id);
    return ResponseEntity.noContent().build(); // 204
}

// GET /api/etudiants/search?nom=diallo
@GetMapping("/search")
public List<Etudiant> rechercher(@RequestParam String nom) {
    return service.rechercherParNom(nom);
}
}

```

✦ TP 09

API REST Complète – Gestion Académique Supdeco

Durée : 50 minutes | Rendu : projet Spring Boot + collection Postman

Travail demandé

47. Créez un projet Spring Boot complet avec Spring Data JPA + MySQL + Spring Web
48. Créez les entités : Etudiant, Filiere, Cours, avec les relations JPA appropriées
49. Créez les repositories Spring Data JPA pour chaque entité
50. Créez les API REST suivantes :

- › GET/POST/PUT/DELETE /api/etudiants
- › GET/POST/PUT/DELETE /api/cours
- › GET /api/filieres avec les étudiants associés
- › GET /api/etudiants/search?nom=XXX
- › GET /api/stats : nombre d'étudiants, moyenne globale, cours total

51. Testez chaque endpoint avec Postman et capturez les résultats
52. BONUS : Ajoutez la gestion des erreurs globale avec @ControllerAdvice et @ExceptionHandler

SÉANCE
10/10
2h

Technologie JEE – L3 Informatique

Projet Intégré – Application Web Spring Boot Complète

Objectifs de la séance

- › Intégrer tous les concepts vus en cours dans une application complète
- › Développer une application Spring Boot full-stack (API + Thymeleaf)
- › Implémenter l'authentification et les rôles utilisateurs
- › Déployer l'application et préparer la soutenance

PROJET FINAL

Contexte du Projet

Système de Gestion Académique – Supdeco Dakar

Vous développerez une application web complète de gestion académique pour Supdeco Dakar. L'application doit permettre la gestion des étudiants, des cours, des inscriptions et des notes, avec une interface moderne et une API REST.

10.1 Sécurité avec Spring Security

```
<!-- pom.xml : ajouter Spring Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/js/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/**").hasAnyRole("ADMIN", "ENSEIGNANT")
                .anyRequest().authenticated()
            )
    }
}
```



```

        .formLogin(form -> form
            .loginPage("/connexion")
            .defaultSuccessUrl("/tableau-de-bord")
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/deconnexion")
            .logoutSuccessUrl("/connexion?logout")
        );
    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

10.2 Cahier des Charges du Projet Final

Fonctionnalités obligatoires (note sur 20) :

Fonctionnalité	Points	Technologie
Structure du projet (packages, architecture MVC)	2 pts	Spring Boot
Entités JPA + relations (Etudiant, Cours, Inscription, Note)	3 pts	JPA/Hibernate
CRUD complet Étudiants (liste, ajout, modif, suppression)	3 pts	Spring MVC
CRUD complet Cours et gestion des inscriptions	2 pts	Spring MVC
Saisie et affichage des notes avec calcul de moyennes	2 pts	Spring MVC
Interface web moderne avec Bootstrap (responsive)	2 pts	Thymeleaf + CSS
Authentification (connexion/déconnexion) + 2 rôles	3 pts	Spring Security
Documentation du code + rapport technique	3 pts	Javadoc + PDF

✦ TP 10

Soutenance & Démo du Projet Final

Durée : Soutenance de 15 min par groupe (2-3 étudiants max)

Livrables à rendre

53. Code source complet sur GitHub (lien à partager avant la séance)
54. Script SQL de la base de données (avec données de test)
55. Rapport technique (PDF, 10-15 pages) : analyse, architecture, diagrammes UML, captures d'écran
56. Présentation PowerPoint (5-8 slides) pour la démo

Déroulement de la soutenance

- › 2 min : Présentation du projet et de l'équipe
- › 5 min : Démonstration live de l'application
- › 5 min : Explication du code et des choix techniques
- › 3 min : Questions du jury

BONUS – Fonctionnalités avancées (+2 pts chacun) :

- › Export PDF des bulletins de notes (ReportLab / iText)
- › API REST complète documentée avec Swagger/OpenAPI
- › Envoi d'email de confirmation d'inscription (Spring Mail)
- › Déploiement Docker (Dockerfile + docker-compose.yml)
- › Tests unitaires avec JUnit 5 (couverture $\geq 60\%$)

RÉCAPITULATIF DU COURS

Séance	Durée	Contenu	TP
01	2h	Rappels Java SE & POO avancée	Système gestion étudiants
02	2h	Architecture JEE & Tomcat	Déploiement application web
03	2h	Servlets HTTP (GET/POST/Session)	App gestion contacts
04	2h	Modèle MVC avec Servlets	App MVC – gestion cours
05	2h	Pages JSP & JSTL	Interface Bootstrap complète
06	2h	Connexion JDBC – CRUD MySQL	CRUD étudiants en BDD
07	2h	ORM & JPA/Hibernate	Mapping entités et relations
08	2h	Introduction Spring Boot	Premier projet Spring Boot
09	2h	Spring Boot – REST API	API REST CRUD complète
10	2h	Projet intégré – Spring Security	Soutenance du projet final

Barème d'évaluation final :

- › 40% – Évaluation continue (TPs notés à chaque séance)
- › 60% – Examen final en présentiel (QCM + questions de code)